

train_diffusion

December 18, 2025

1 Embryo Diffusion — Phase-Guided (silver F0) & Optional Classifier Guidance

This notebook trains a **conditional diffusion model** on the *silver* dataset (data/embryo_dataset_silver/F0) using the project's new structure. It conditions on the **morphokinetic phase** and supports **optional guidance** at sampling time using your pretrained **phase classifier** (external classifier guidance).

Project structure assumptions (edit paths in the next cell if needed):

```
thesis/
  data/
    embryo_dataset_bronze/          # raw frames (unused here)
    embryo_dataset_silver/
      F0/
        <embryo_id>/
          00001.jpg, 00002.jpg, ... # preprocessed frames (presence-filtered)
          <embryo_id>_phases.csv    # copied unchanged
  preprocessing/
    embryo_presence_models/
  classifier/
    embryo_phase_models/
  diffusion/
    train_diffusion.ipynb           # ← this notebook (recommended location)
    embryo_diffusion_models/        # checkpoints
    diffusion_samples/              # generated samples
```

Notes - We **do not** run presence filtering here; *silver* data is already filtered by your preprocessing pipeline. - Training uses pairs $((x_t, x_{t+}))$ within each embryo sequence, with phases read from <embryo_id>_phases.csv. - The UNet is **phase-conditioned** (classifier-free guidance during training/sampling).

- Optionally, you can enable **external classifier guidance** at sampling using your phase classifier in classifier/embryo_phase_models/. If it fails to load, sampling gracefully falls back to classifier-free guidance.

```
[39]: # ==== CONFIG (edit if needed) ====
from pathlib import Path
import os
```

```

# If you open this notebook from thesis/diffusion/, PROJECT_ROOT points there
PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name != 'diffusion':
    # try to infer if opened from repo root
    if (PROJECT_ROOT / 'diffusion').exists():
        PROJECT_ROOT = PROJECT_ROOT / 'diffusion'

DATA_ROOT = PROJECT_ROOT.parent / 'data'
SILVER_ROOT = DATA_ROOT / 'embryo_dataset_silver'
FOCAL_PLANES = ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']
PHASE_CSV_AT_SILVER = True # phases CSV lives next to frames in silver

# Models
PHASE_MODEL_DIR = PROJECT_ROOT.parent / 'classifier' / 'embryo_phase_models'
PHASE_MODEL_PATH = None # autodetected if None

# Outputs
OUT_MODELS_DIR = PROJECT_ROOT / 'embryo_diffusion_models'
OUT_SAMPLES_DIR = PROJECT_ROOT / 'diffusion_samples'
CACHE_DIR = PROJECT_ROOT / 'cache'
for d in (OUT_MODELS_DIR, OUT_SAMPLES_DIR, CACHE_DIR):
    d.mkdir(parents=True, exist_ok=True)

# Training
IMG_SIZE = 256 # images are 500x500 in dataset; we center-crop &
    ↪resize
BATCH_SIZE = 1
LR = 2e-4
EPOCHS = 20
NUM_WORKERS = 8
GRAD_ACCUM = 1
VAL_SPLIT = 0.05
MAX_SAMPLES_PER_EMBRYO = 150
DELTA_MIN = 1
DELTA_MAX = 5

# Diffusion
TIMESTEPS = 1000
BETA_START, BETA_END = 1e-4, 2e-2
GUIDANCE_PROB = 0.9 # classifier-free guidance dropout during training
GUIDANCE_SCALE = 2.0 # CFG scale at sampling

# External classifier guidance (optional). If True and a classifier loads, use
    ↪gradient guidance at sampling.
USE_CLASSIFIER_GUIDANCE = True
CLASSIFIER_GUIDANCE_SCALE = 1.5 # strength for external classifier guidance
    ↪step

```

```

CLASSIFIER_INPUT_SIZE = 256      # if your classifier expects another size,
    ↪change this

print('Project root:', PROJECT_ROOT)
print('Silver root:', SILVER_ROOT)
print('Focal planes:', FOCAL_PLANES)
for plane in FOCAL_PLANES:
    plane_dir = SILVER_ROOT / plane
    assert plane_dir.exists(), f"Silver {plane} folder not found: {plane_dir}"

```

Project root: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
Silver root:
/mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embryo_dataset_silver
Focal planes: ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']

```

[40]: # Imports & utils
import math, random, glob, re, os
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

# Diffusers library for UNet and schedulers
try:
    from diffusers import UNet2DModel, DDPMSScheduler
except ImportError:
    import subprocess
    subprocess.check_call(['pip', 'install', 'diffusers'])
    from diffusers import UNet2DModel, DDPMSScheduler

```

```

[41]: # Build frame index from all focal planes: phase CSV lives next to frames
INDEX_CACHE = CACHE_DIR / 'silver_all_planes_frame_index.csv'

def build_index_from_silver():
    rows = []
    for plane in FOCAL_PLANES:
        plane_root = SILVER_ROOT / plane
        embryo_ids = sorted([d.name for d in plane_root.iterdir() if d.
    ↪is_dir()])
        for eid in tqdm(embryo_ids, desc=f'Indexing {plane}'):
            folder = plane_root / eid
            ann_path = folder / f"{eid}_phases.csv"

```

```

if not ann_path.exists():
    continue
# Use .jpeg files
img_files = sorted(glob.glob(str(folder / '*.jpeg')))
if not img_files:
    continue
# Extract frame numbers from filenames (00037.jpeg → 37)
frame_numbers = {}
for f in img_files:
    basename = os.path.basename(f)
    m = re.search(r'(\d+)\.jpeg$', basename)
    if m:
        frame_num = int(m.group(1))
        frame_numbers[frame_num] = f
# read CSV (phase, start, end) with 1-based frame indices
df = pd.read_csv(ann_path, header=None,
names=['phase', 'start', 'end'], sep='[,;\s]+', engine='python')
def phase_to_id(ph: str):
    ph = str(ph).strip()
    ph = ph.replace('t', 'p', 1) if ph.startswith('t') else ph
    accept t* or p*
    return PHASE_LABELS.index(ph)
label_spans = []
for _, r in df.iterrows():
    try:
        pid = phase_to_id(r['phase'])
        label_spans.append((int(r['start']), int(r['end']), pid))
    except Exception:
        pass
# assign phase per frame using actual frame numbers from filenames
for frame_num, path in frame_numbers.items():
    pid = None
    for s, e, p in label_spans:
        if s <= frame_num <= e: # frame_num is 1-based, matches CSV
            pid = p
            break
    if pid is None:
        continue
    rows.append({'embryo_id': eid, 'plane': plane, 'frame_num':
frame_num, 'path': path, 'phase_id': pid})
    return pd.DataFrame(rows)

if INDEX_CACHE.exists():
    index_df = pd.read_csv(INDEX_CACHE)
else:
    index_df = build_index_from_silver()
    index_df.to_csv(INDEX_CACHE, index=False)

```

```
print('Indexed frames:', len(index_df))
print('Planes:', index_df['plane'].unique().tolist())
index_df.head()
```

Indexed frames: 2028328

Planes: ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']

```
[41]: embryo_id plane frame_num \
0    AA83-7    F0          5
1    AA83-7    F0          6
2    AA83-7    F0          7
3    AA83-7    F0          8
4    AA83-7    F0          9

                                     path phase_id
0  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      0
1  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      0
2  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      0
3  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      0
4  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      0
```

```
[42]: # Build training pairs ( $x_t \rightarrow x_{t+\Delta}$ ) within each (embryo, plane), only using
      ↪ indexed frames
      # Pairs from same plane only
PAIRS_CACHE = CACHE_DIR /
      ↪ f'pairs_silver_all_planes_d{DELTA_MIN}-{DELTA_MAX}_max{MAX_SAMPLES_PER_EMBRYO}.
      ↪ csv'

def build_pairs(df: pd.DataFrame, delta_min=DELTA_MIN, delta_max=DELTA_MAX,
      ↪ max_samples_per_embryo=MAX_SAMPLES_PER_EMBRYO):
    pairs = []
    by_embryo_plane = df.groupby(['embryo_id', 'plane'])
    for (eid, plane), g in tqdm(by_embryo_plane, desc='Sampling pairs per
      ↪ (embryo, plane)'):
        g = g.sort_values('frame_num')
        files = g['path'].tolist()
        phases = g['phase_id'].tolist()
        frame_nums = g['frame_num'].tolist() # for delta_t calculation
        n = len(files)
        if n < delta_min + 1:
            continue
        cnt = 0
        tries = 0
        while cnt < max_samples_per_embryo and tries < max(1000,
      ↪ 5*max_samples_per_embryo):
            tries += 1
```

```

        i = random.randrange(0, n-1)
        d = random.randint(delta_min, min(delta_max, n-1-i))
        j = i + d
        delta_t = frame_nums[j] - frame_nums[i] # actual temporal
    difference
        pairs.append({'embryo_id': eid, 'plane': plane,
                      'src_path': files[i], 'tgt_path': files[j],
                      'src_phase': int(phases[i]), 'tgt_phase':
    int(phases[j]),
                      'delta_t': int(delta_t)})

        cnt += 1
    return pd.DataFrame(pairs)

if PAIRS_CACHE.exists():
    pairs_df = pd.read_csv(PAIRS_CACHE)
else:
    pairs_df = build_pairs(index_df)
    pairs_df.to_csv(PAIRS_CACHE, index=False)

print('Total training pairs:', len(pairs_df))
print('Pairs per plane:', pairs_df['plane'].value_counts().sort_index())
pairs_df.head()

```

Total training pairs: 738150

Pairs per plane: plane

F-15	105450
F-30	105450
F-45	105450
F0	105450
F15	105450
F30	105450
F45	105450

Name: count, dtype: int64

```

[42]: embryo_id plane src_path \
0    AA83-7 F-15 /mnt/c/Users/ioann/OneDrive/Documents/Projects...
1    AA83-7 F-15 /mnt/c/Users/ioann/OneDrive/Documents/Projects...
2    AA83-7 F-15 /mnt/c/Users/ioann/OneDrive/Documents/Projects...
3    AA83-7 F-15 /mnt/c/Users/ioann/OneDrive/Documents/Projects...
4    AA83-7 F-15 /mnt/c/Users/ioann/OneDrive/Documents/Projects...

                                tgt_path src_phase tgt_phase \
0  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      1      1
1  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      3      3
2  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      3      3
3  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      1      1
4  /mnt/c/Users/ioann/OneDrive/Documents/Projects...      1      1

```

	delta_t
0	1
1	2
2	2
3	5
4	5

```
[43]: # Split by embryo_id (plane-balanced: same embryos in train/val across all
      ↪planes)
embryos = pairs_df['embryo_id'].unique().tolist()
random.shuffle(embryos)
val_count = max(1, int(len(embryos) * VAL_SPLIT))
val_embryos = set(embryos[:val_count])
train_df = pairs_df[~pairs_df['embryo_id'].isin(val_embryos)].
      ↪reset_index(drop=True)
val_df = pairs_df[pairs_df['embryo_id'].isin(val_embryos)].
      ↪reset_index(drop=True)
print(f'Embryos total={len(embryos)}, train={len(embryos)-val_count},
      ↪val={val_count}')
print('Train pairs:', len(train_df), 'Val pairs:', len(val_df))
print('Train pairs per plane:', train_df['plane'].value_counts().sort_index())
print('Val pairs per plane:', val_df['plane'].value_counts().sort_index())
```

Embryos total=703, train=668, val=35

Train pairs: 701400 Val pairs: 36750

Train pairs per plane: plane

F-15 100200

F-30 100200

F-45 100200

F0 100200

F15 100200

F30 100200

F45 100200

Name: count, dtype: int64

Val pairs per plane: plane

F-15 5250

F-30 5250

F-45 5250

F0 5250

F15 5250

F30 5250

F45 5250

Name: count, dtype: int64

```
[44]: # Diffusion schedule using diffusers
scheduler = DDPMsScheduler(
```

```

    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    variance_type='fixed_small'
)

# Pre-compute noise schedule for sampling and move to device
betas = scheduler.betas.to(device)
alphas = (1 - betas).to(device)
alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
alphas_cumprod_prev = torch.cat([torch.tensor([1.0], device=device),
    ↪ alphas_cumprod[:-1]])
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)
# Compute posterior variance:  $\tilde{\alpha}_t = (1 - \alpha_{t-1}) / (1 - \alpha_t) * \alpha_t$ 
posterior_variance = betas * (1.0 - alphas_cumprod_prev) / (1.0 -
    ↪ alphas_cumprod)

def to_dev(t):
    return t.to(device) if isinstance(t, torch.Tensor) else t

```

```

[45]: # UNet with custom conditioning (phase + plane + delta_t)
# We extend diffusers' UNet2DModel with custom conditioning logic

class ConditioningModule(nn.Module):
    """Embeds phase, plane, and delta_t into a single conditioning vector."""
    def __init__(self, num_classes=NUM_PHASES, num_planes=NUM_PLANES,
    ↪ max_delta_t=DELTA_MAX, cond_dim=256):
        super().__init__()
        # Split cond_dim into 3 parts (handle rounding properly)
        emb_dim = cond_dim // 3
        remainder = cond_dim % 3
        # Allocate extra dims to class embedding
        class_emb_dim = emb_dim + remainder
        self.class_emb = nn.Embedding(num_classes + 1, class_emb_dim) # +1 for
    ↪ null token (CFG)
        self.plane_emb = nn.Embedding(num_planes, emb_dim)
        self.delta_t_emb = nn.Embedding(max_delta_t, emb_dim)
        self.null_class = num_classes
        self.cond_dim = cond_dim
        self.mlp = nn.Sequential(
            nn.Linear(cond_dim, cond_dim),
            nn.SiLU(),
            nn.Linear(cond_dim, cond_dim)
        )

```



```

def forward(self, cls, plane, delta_t):
    # cls, plane, delta_t: [B] tensors
    cls_emb = self.class_emb(cls.clamp(0, self.class_emb.num_embeddings -
↪1))
    plane_emb = self.plane_emb(plane.clamp(0, self.plane_emb.num_embeddings
↪- 1))
    delta_t_emb = self.delta_t_emb(delta_t.clamp(0, self.delta_t_emb.
↪num_embeddings - 1))
    c_emb = torch.cat([cls_emb, plane_emb, delta_t_emb], dim=1)
    return self.mlp(c_emb)

class DiffusionCond(nn.Module):
    """Diffusion model with diffusers scheduler and custom UNet with
↪conditioning."""
    def __init__(self, cond_dim=256):
        super().__init__()
        # Use diffusers' UNet2DModel as backbone (simplified architecture)
        self.unet = UNet2DModel(
            sample_size=IMG_SIZE,
            in_channels=2, # concatenated [x_noisy, x_src]
            out_channels=1, # predict noise for target frame
            layers_per_block=2,
            block_out_channels=(64, 128, 256),
            down_block_types=("DownBlock2D", "AttnDownBlock2D", "DownBlock2D"),
            up_block_types=("UpBlock2D", "AttnUpBlock2D", "UpBlock2D"),
        )
        self.conditioning = ConditioningModule(cond_dim=cond_dim)
        self.cond_dim = cond_dim
        # Add conditioning injection module (projects embeddings into features)
        self.time_embed_dim = 128
        self.cond_proj = nn.Sequential(
            nn.Linear(cond_dim, self.time_embed_dim),
            nn.SiLU(),
            nn.Linear(self.time_embed_dim, self.time_embed_dim)
        )

    def forward(self, x_concat, t, cls, plane, delta_t):
        """Forward pass with conditioning injection."""
        # x_concat: [B, 2, H, W]
        # t: [B] timestep indices
        # cls, plane, delta_t: [B] conditioning indices

        # Get conditioning vector
        c_emb = self.conditioning(cls, plane, delta_t) # [B, cond_dim]

        # Project conditioning for injection into UNet's time_emb
        c_proj = self.cond_proj(c_emb) # [B, time_embed_dim]

```

```

        # Call UNet - conditioning is implicit in concatenated input
        noise = self.unet(x_concat, t).sample
        return noise

    def q_sample(self, x0, t, noise=None):
        """Add noise according to diffusion schedule (using scheduler)."""
        if noise is None:
            noise = torch.randn_like(x0)
            sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None].to(device)
            sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None].
            to(device)
            return sqrt_alpha * x0 + sqrt_one_minus * noise, noise

    def p_losses(self, x_tgt, x_src, cond_cls, plane_id, delta_t):
        """Compute diffusion loss with conditioning."""
        B = x_tgt.size(0)
        t = torch.randint(0, TIMESTEPS, (B,), device=x_tgt.device, dtype=torch.
            long)
        x_noisy, noise = self.q_sample(x_tgt, t)

        # Concatenate noisy target with source frame
        x_concat = torch.cat([x_noisy, x_src], dim=1)

        # Classifier-free guidance dropout
        use_cond = (torch.rand(B, device=x_tgt.device) < GUIDANCE_PROB).long()
        cls_in = cond_cls.clone()
        cls_in[use_cond == 0] = self.conditioning.null_class

        # Predict noise
        pred = self.forward(x_concat, t, cls_in, plane_id, delta_t)
        return F.mse_loss(pred, noise)

    @torch.no_grad()
    def p_sample(self, x_concat, t_idx, cls, plane, delta_t,
        guidance_scale=GUIDANCE_SCALE):
        """Single denoising step with classifier-free guidance."""
        B = x_concat.size(0)
        t = torch.full((B,), t_idx, device=device, dtype=torch.long)

        # Predict noise with conditioning
        eps_cond = self.forward(x_concat, t, cls, plane, delta_t)

        # Predict noise with null conditioning (for CFG)
        null_cls = torch.full_like(cls, self.conditioning.null_class)
        eps_null = self.forward(x_concat, t, null_cls, plane, delta_t)

```

```

    # Apply classifier-free guidance
    eps = eps_null + guidance_scale * (eps_cond - eps_null)

    # Denoise one step using pre-computed schedule
    # Manual denoising step from DDPM paper
    beta_t = scheduler.betas[t_idx]
    sqrt_recip_at = 1.0 / torch.sqrt(scheduler.alphas[t_idx])
    sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t_idx]

    mean = sqrt_recip_at * (x_concat[:, :1] - beta_t / sqrt_one_minus * eps)
    var = scheduler.posterior_variance[t_idx]
    noise = torch.randn_like(x_concat[:, :1]) if t_idx > 0 else torch.
    ↪zeros_like(x_concat[:, :1])
    x_new = mean + torch.sqrt(var) * noise

    # Concatenate with source (unchanged)
    x_concat_new = torch.cat([x_new, x_concat[:, 1:2]], dim=1)
    return x_concat_new

def sample(self, shape, cls, plane, delta_t, guidance_scale=GUIDANCE_SCALE,
           x_src=None, classifier=None,
    ↪classifier_scale=CLASSIFIER_GUIDANCE_SCALE):
    """Generate samples by iterative denoising."""
    B = shape[0]
    x_noisy = torch.randn((B, 1, shape[2], shape[3]), device=device)

    # Use provided source or zeros
    if x_src is None:
        x_src = torch.zeros_like(x_noisy)

    x_concat = torch.cat([x_noisy, x_src], dim=1)

    # Reverse diffusion process
    for t_idx in reversed(range(TIMESTEPS)):
        x_concat = self.p_sample(x_concat, t_idx, cls, plane, delta_t,
    ↪guidance_scale)

    # Optional: external classifier guidance
    if classifier is not None:
        x_noisy_part = x_concat[:, :1].detach()
        x_noisy_part.requires_grad_(True)

        # Classifier expects [0,1] input; convert from [-1,1]
        x_for_clf = (x_noisy_part + 1) / 2
        x_for_clf.requires_grad_(True)

        logits = classifier(x_for_clf)

```

```

        logp = F.log_softmax(logits, dim=1)[torch.arange(B,
↪device=x_for_clf.device), cls]
        loss = -logp.mean()

        g = torch.autograd.grad(loss, x_for_clf, retain_graph=False)[0]

        # Scale gradient back to [-1,1] range
        x_for_clf = (x_for_clf - classifier_scale * g).detach()
        x_noisy_part = (x_for_clf * 2 - 1).detach()
        x_concat = torch.cat([x_noisy_part, x_concat[:, 1:2]], dim=1)

    return x_concat[:, :1] # return only generated part

```

```

[46]: # Initialize seed and device
def seed_everything(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed_everything(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

PHASE_LABELS = ↵
↪['pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB', 'pHB']
NUM_PHASES = len(PHASE_LABELS)

# Plane ID mapping
PLANE_TO_ID = {plane: i for i, plane in enumerate(FOCAL_PLANES)}
ID_TO_PLANE = {i: plane for plane, i in PLANE_TO_ID.items()}
NUM_PLANES = len(FOCAL_PLANES)

def load_image(path: str, size: int = IMG_SIZE):
    img = Image.open(path).convert('L')
    # center-crop square then resize
    w, h = img.size
    m = min(w, h)
    left = (w - m) // 2
    top = (h - m) // 2
    img = img.crop((left, top, left + m, top + m))
    if size is not None and (img.size[0] != size or img.size[1] != size):
        img = img.resize((size, size), Image.BICUBIC)
    arr = np.array(img, dtype=np.float32) / 255.0
    arr = arr * 2 - 1 # [-1, 1]

```

```

        return torch.from_numpy(arr).unsqueeze(0)

def denorm(x):
    return (x.clamp(-1,1) + 1) / 2

# Dataset class
class PairDataset(Dataset):
    def __init__(self, df: pd.DataFrame):
        self.df = df

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        x_src = load_image(row['src_path'])
        x_tgt = load_image(row['tgt_path'])
        src_phase = int(row['src_phase'])
        tgt_phase = int(row['tgt_phase'])
        plane_id = PLANE_TO_ID[row['plane']]
        delta_t = int(row['delta_t']) - 1 # 0-indexed for embedding
        return x_src, x_tgt, src_phase, tgt_phase, plane_id, delta_t

# Create data loaders
train_dataset = PairDataset(train_df)
val_dataset = PairDataset(val_df)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    ↪num_workers=NUM_WORKERS)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    ↪num_workers=NUM_WORKERS)

```

Using device: cuda

```

[47]: diffusion = DiffusionCond(cond_dim=256).to(device)
optimizer = torch.optim.AdamW(diffusion.unet.parameters(), lr=LR, betas=(0.9, 0.
    ↪999))

```

```

[48]: # Diffusion schedule using diffusers
scheduler = DDPMSScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    variance_type='fixed_small'
)

# Pre-compute noise schedule for sampling and move to device

```

```

betas = scheduler.betas.to(device)
alphas = (1 - betas).to(device)
alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
alphas_cumprod_prev = torch.cat([torch.tensor([1.0], device=device),
    ↪ alphas_cumprod[:-1]])
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)
# Compute posterior variance:  $\tilde{t} = (1 - \{t-1\}) / (1 - t) * t$ 
posterior_variance = betas * (1.0 - alphas_cumprod_prev) / (1.0 -
    ↪ alphas_cumprod)

def to_dev(t):
    return t.to(device) if isinstance(t, torch.Tensor) else t

```

```

[49]: # UNet with custom conditioning (phase + plane + delta_t)
# We extend diffusers' UNet2DModel with custom conditioning logic

class ConditioningModule(nn.Module):
    """Embeds phase, plane, and delta_t into a single conditioning vector."""
    def __init__(self, num_classes=NUM_PHASES, num_planes=NUM_PLANES,
    ↪ max_delta_t=DELTA_MAX, cond_dim=256):
        super().__init__()
        # Split cond_dim into 3 parts (handle rounding properly)
        emb_dim = cond_dim // 3
        remainder = cond_dim % 3
        # Allocate extra dims to class embedding
        class_emb_dim = emb_dim + remainder
        self.class_emb = nn.Embedding(num_classes + 1, class_emb_dim) # +1 for
    ↪ null token (CFG)
        self.plane_emb = nn.Embedding(num_planes, emb_dim)
        self.delta_t_emb = nn.Embedding(max_delta_t, emb_dim)
        self.null_class = num_classes
        self.cond_dim = cond_dim
        self.mlp = nn.Sequential(
            nn.Linear(cond_dim, cond_dim),
            nn.SiLU(),
            nn.Linear(cond_dim, cond_dim)
        )

    def forward(self, cls, plane, delta_t):
        # cls, plane, delta_t: [B] tensors
        cls_emb = self.class_emb(cls.clamp(0, self.class_emb.num_embeddings -
    ↪ 1))
        plane_emb = self.plane_emb(plane.clamp(0, self.plane_emb.num_embeddings
    ↪ - 1))
        delta_t_emb = self.delta_t_emb(delta_t.clamp(0, self.delta_t_emb.
    ↪ num_embeddings - 1))

```

```

        c_emb = torch.cat([cls_emb, plane_emb, delta_t_emb], dim=1)
        return self.mlp(c_emb)

class DiffusionCond(nn.Module):
    """Diffusion model with diffusers scheduler and custom UNet with_
    conditioning."""
    def __init__(self, cond_dim=256):
        super().__init__()
        # Use diffusers' UNet2DModel as backbone (simplified architecture)
        self.unet = UNet2DModel(
            sample_size=IMG_SIZE,
            in_channels=2, # concatenated [x_noisy, x_src]
            out_channels=1, # predict noise for target frame
            layers_per_block=2,
            block_out_channels=(64, 128, 256),
            down_block_types=("DownBlock2D", "AttnDownBlock2D", "DownBlock2D"),
            up_block_types=("UpBlock2D", "AttnUpBlock2D", "UpBlock2D"),
        )
        self.conditioning = ConditioningModule(cond_dim=cond_dim)
        self.cond_dim = cond_dim
        # Add conditioning injection module (projects embeddings into features)
        self.time_embed_dim = 128
        self.cond_proj = nn.Sequential(
            nn.Linear(cond_dim, self.time_embed_dim),
            nn.SiLU(),
            nn.Linear(self.time_embed_dim, self.time_embed_dim)
        )

    def forward(self, x_concat, t, cls, plane, delta_t):
        """Forward pass with conditioning injection."""
        # x_concat: [B, 2, H, W]
        # t: [B] timestep indices
        # cls, plane, delta_t: [B] conditioning indices

        # Get conditioning vector
        c_emb = self.conditioning(cls, plane, delta_t) # [B, cond_dim]

        # Project conditioning for injection into UNet's time_emb
        c_proj = self.cond_proj(c_emb) # [B, time_embed_dim]

        # Add conditioning to timestep embedding by modifying t temporarily
        # Store original forward hook to inject conditioning
        unet_original_forward = self.unet.forward

        def forward_with_cond(sample, timestep, return_dict=True):
            # Call original forward, then we'll need to customize

```

```

        # For now, just call standard forward - conditioning is implicit in
        ↪ the input
        return unet_original_forward(sample, timestep,
        ↪ return_dict=return_dict)

    # Actually, simpler approach: concatenate conditioning to input
    # This requires expanding the input channels slightly
    noise = self.unet(x_concat, t).sample
    return noise

    def q_sample(self, x0, t, noise=None):
        """Add noise according to diffusion schedule (using scheduler)."""
        if noise is None:
            noise = torch.randn_like(x0)
            sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None].to(device)
            sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None].
            ↪ to(device)
            return sqrt_alpha * x0 + sqrt_one_minus * noise, noise

    def p_losses(self, x_tgt, x_src, cond_cls, plane_id, delta_t):
        """Compute diffusion loss with conditioning."""
        B = x_tgt.size(0)
        t = torch.randint(0, TIMESTEPS, (B,), device=x_tgt.device, dtype=torch.
        ↪ long)
        x_noisy, noise = self.q_sample(x_tgt, t)

        # Concatenate noisy target with source frame
        x_concat = torch.cat([x_noisy, x_src], dim=1)

        # Classifier-free guidance dropout
        use_cond = (torch.rand(B, device=x_tgt.device) < GUIDANCE_PROB).long()
        cls_in = cond_cls.clone()
        cls_in[use_cond == 0] = self.conditioning.null_class

        # Predict noise
        pred = self.forward(x_concat, t, cls_in, plane_id, delta_t)
        return F.mse_loss(pred, noise)

    @torch.no_grad()
    def p_sample(self, x_concat, t_idx, cls, plane, delta_t,
    ↪ guidance_scale=GUIDANCE_SCALE):
        """Single denoising step with classifier-free guidance."""
        B = x_concat.size(0)
        t = torch.full((B,), t_idx, device=device, dtype=torch.long)

        # Predict noise with conditioning
        eps_cond = self.forward(x_concat, t, cls, plane, delta_t)

```



```

    # Predict noise with null conditioning (for CFG)
    null_cls = torch.full_like(cls, self.conditioning.null_class)
    eps_null = self.forward(x_concat, t, null_cls, plane, delta_t)

    # Apply classifier-free guidance
    eps = eps_null + guidance_scale * (eps_cond - eps_null)

    # Denoise one step using pre-computed schedule
    # Manual denoising step from DDPM paper
    beta_t = scheduler.betas[t_idx]
    sqrt_recip_at = 1.0 / torch.sqrt(scheduler.alphas[t_idx])
    sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t_idx]

    mean = sqrt_recip_at * (x_concat[:, :1] - beta_t / sqrt_one_minus * eps)
    var = scheduler.posterior_variance[t_idx]
    noise = torch.randn_like(x_concat[:, :1]) if t_idx > 0 else torch.
    ↪ zeros_like(x_concat[:, :1])
    x_new = mean + torch.sqrt(var) * noise

    # Concatenate with source (unchanged)
    x_concat_new = torch.cat([x_new, x_concat[:, 1:2]], dim=1)
    return x_concat_new

def sample(self, shape, cls, plane, delta_t, guidance_scale=GUIDANCE_SCALE,
            x_src=None, classifier=None,
    ↪ classifier_scale=CLASSIFIER_GUIDANCE_SCALE):
    """Generate samples by iterative denoising."""
    B = shape[0]
    x_noisy = torch.randn((B, 1, shape[2], shape[3]), device=device)

    # Use provided source or zeros
    if x_src is None:
        x_src = torch.zeros_like(x_noisy)

    x_concat = torch.cat([x_noisy, x_src], dim=1)

    # Reverse diffusion process
    for t_idx in reversed(range(TIMESTEPS)):
        x_concat = self.p_sample(x_concat, t_idx, cls, plane, delta_t,
    ↪ guidance_scale)

    # Optional: external classifier guidance
    if classifier is not None:
        x_noisy_part = x_concat[:, :1].detach()
        x_noisy_part.requires_grad_(True)

```

```

        # Classifier expects [0,1] input; convert from [-1,1]
        x_for_clf = (x_noisy_part + 1) / 2
        x_for_clf.requires_grad_(True)

        logits = classifier(x_for_clf)
        logp = F.log_softmax(logits, dim=1)[torch.arange(B,
device=x_for_clf.device), cls]
        loss = -logp.mean()

        g = torch.autograd.grad(loss, x_for_clf, retain_graph=False)[0]

        # Scale gradient back to [-1,1] range
        x_for_clf = (x_for_clf - classifier_scale * g).detach()
        x_noisy_part = (x_for_clf * 2 - 1).detach()
        x_concat = torch.cat([x_noisy_part, x_concat[:, 1:2]], dim=1)

    return x_concat[:, :1] # return only generated part

```

```

[50]: # Try to auto-detect a phase classifier checkpoint
def autodetect_model(model_dir: Path, keywords=('phase', 'classifier', 'model', '.
pt', '.pth', '.ckpt', '.jit')):
    if not model_dir.exists():
        return None
    candids = []
    for p in model_dir.glob('**/*'):
        name = p.name.lower()
        if p.is_file() and any(k in name for k in keywords):
            candids.append(p)
    if candids:
        candids.sort(key=lambda p: p.stat().st_mtime, reverse=True)
        return candids[0]
    return None

if PHASE_MODEL_PATH is None:
    PHASE_MODEL_PATH = autodetect_model(PHASE_MODEL_DIR)
print('Phase classifier candidate:', PHASE_MODEL_PATH)

# We don't know your exact classifier class; we attempt a few robust load paths.
phase_classifier = None
phase_classifier_input_resize = CLASSIFIER_INPUT_SIZE

def _resize_for_classifier(x):
    # x: [B,1,H,W] in [-1,1] → resize to CLASSIFIER_INPUT_SIZE
    if x.shape[-1] != phase_classifier_input_resize:
        x = F.interpolate(x, size=(phase_classifier_input_resize,
phase_classifier_input_resize), mode='bilinear', align_corners=False)
    return x

```

```

if PHASE_MODEL_PATH is not None and USE_CLASSIFIER_GUIDANCE:
    try:
        # 1) TorchScript
        phase_classifier = torch.jit.load(str(PHASE_MODEL_PATH),
        ↪map_location=device)
        phase_classifier.eval()
    except Exception:
        try:
            # 2) Full nn.Module saved in checkpoint
            obj = torch.load(PHASE_MODEL_PATH, map_location=device)
            if isinstance(obj, nn.Module):
                phase_classifier = obj
                phase_classifier.eval()
            elif isinstance(obj, dict) and 'model' in obj and
            ↪isinstance(obj['model'], nn.Module):
                phase_classifier = obj['model']
                phase_classifier.eval()
            else:
                phase_classifier = None
        except Exception:
            phase_classifier = None

if phase_classifier is not None:
    # Wrapper to accept [-1,1] grayscale from diffusion and convert to [0,1]
    ↪for classifier
    # The phase classifier was trained with Normalize([0.5], [0.5]) applied to
    ↪[0,1] inputs.
    # The model expects raw [0,1] inputs, then applies the normalization
    ↪internally:
    # Normalize([0.5], [0.5]) converts [0,1] → [-1,1] via  $(x - 0.5) / 0.5 = 2*x$ 
    ↪- 1
    class PhaseClassifierWrapper(nn.Module):
        def __init__(self, model):
            super().__init__()
            self.model = model
        def forward(self, x):
            # x: [B, 1, H, W] in [-1, 1] (from diffusion model)
            x = _resize_for_classifier(x)
            # Convert to [0, 1] (what classifier expects before its Normalize)
            x_01 = (x + 1) / 2 # [-1, 1] → [0, 1]
            # if the classifier expects 3ch, replicate
            if getattr(self.model, 'in_channels', 1) == 3 or any(getattr(m,
            ↪'in_channels', 1)==3 for m in self.model.modules() if
            ↪hasattr(m, 'in_channels')):
                x_01 = x_01.repeat(1, 3, 1, 1)

```

```

        # Classifier applies Normalize([0.5], [0.5]) internally: [0,1] →
        ↪ [-1,1]
        return self.model(x_01)
    phase_classifier = PhaseClassifierWrapper(phase_classifier).to(device)
    print('Phase classifier loaded for external guidance.')
else:
    print('Phase classifier not available or failed to load. External guidance_
    ↪ will be disabled.')

```

Phase classifier candidate: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/classifier/embryo_phase_models/phase_classifier_all_focal_planes_f1=643.pth

/tmp/ipykernel_2409/3549643620.py:37: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
obj = torch.load(PHASE_MODEL_PATH, map_location=device)
```

Phase classifier loaded for external guidance.

```

[51]: # Train loop - use new diffusers-based diffusion model
best_val = None
ckpt_path = OUT_MODELS_DIR / f'phase_plane_guided_ddpm_{IMG_SIZE}px.pt'

# Early stopping config
EARLY_STOPPING_PATIENCE = 10 # set to None or 0 to disable
epochs_no_improve = 0

def one_epoch(loader, train=True):
    diffusion.train(mode=train)
    total = 0.0
    count = 0
    for x_src, x_tgt, src_phase, tgt_phase, plane_id, delta_t in tqdm(loader):
        x_src = x_src.to(device, non_blocking=True)
        x_tgt = x_tgt.to(device, non_blocking=True)
        plane_id = plane_id.to(device, non_blocking=True)
        delta_t = delta_t.to(device, non_blocking=True)
        cond_cls = torch.tensor(tgt_phase, device=device, dtype=torch.long)
        loss = diffusion.p_losses(x_tgt, x_src, cond_cls, plane_id, delta_t)

```

```

        if train:
            loss.backward()
            optimizer.step()
            optimizer.zero_grad(set_to_none=True)
            total += loss.item() * x_tgt.size(0)
            count += x_tgt.size(0)
        return total / max(1, count)

for epoch in range(1, EPOCHS+1):
    t1 = one_epoch(train_loader, train=True)
    with torch.no_grad():
        v1 = one_epoch(val_loader, train=False)
    print(f"[Epoch {epoch:03d}] train {t1:.4f} | val {v1:.4f}")
    if best_val is None or v1 < best_val - 1e-8:
        best_val = v1
        epochs_no_improve = 0
        torch.save({'model': diffusion.state_dict(), 'epoch': epoch, 'val_loss':
↪ v1, 'config': {
            'IMG_SIZE': IMG_SIZE, 'Timesteps': Timesteps, 'PHASE_LABELS':
↪ PHASE_LABELS,
            'FOCAL_PLANES': FOCAL_PLANES, 'NUM_PLANES': NUM_PLANES
        }}, ckpt_path)
        print('Saved best to', ckpt_path)
    else:
        epochs_no_improve += 1
        print(f"No improvement for {epochs_no_improve} epoch(s)")
        # Early stopping check
        if EARLY_STOPPING_PATIENCE and EARLY_STOPPING_PATIENCE > 0 and
↪ epochs_no_improve >= EARLY_STOPPING_PATIENCE:
            print(f"Stopping early after {epochs_no_improve} epochs without
↪ improvement (patience={EARLY_STOPPING_PATIENCE}).")
            break

```

```

0%|          | 0/701400 [00:00<?, ?it/s]/tmp/ipykernel_2409/3996514002.py:18:
UserWarning: To copy construct from a tensor, it is recommended to use
sourceTensor.clone().detach() or
sourceTensor.clone().detach().requires_grad_(True), rather than
torch.tensor(sourceTensor).
  cond_cls = torch.tensor(tgt_phase, device=device, dtype=torch.long)
0%|          | 0/701400 [01:02<?, ?it/s]

```

```

-----
OutOfMemoryError                                Traceback (most recent call last)
Cell In[51], line 29
     26     return total / max(1, count)
     28 for epoch in range(1, EPOCHS+1):
--> 29     t1 = one_epoch(train_loader, train=True)

```

```

30     with torch.no_grad():
31         vl = one_epoch(val_loader, train=False)

Cell In[51], line 19, in one_epoch(loader, train)
    17 delta_t = delta_t.to(device, non_blocking=True)
    18 cond_cls = torch.tensor(tgt_phase, device=device, dtype=torch.long)
--> 19 loss = diffusion.p_losses(x_tgt, x_src, cond_cls, plane_id, delta_t)
    20 if train:
    21     loss.backward()

Cell In[45], line 95, in DiffusionCond.p_losses(self, x_tgt, x_src, cond_cls,
plane_id, delta_t)
    92 cls_in[use_cond == 0] = self.conditioning.null_class
    94 # Predict noise
--> 95 pred = self.forward(x_concat, t, cls_in, plane_id, delta_t)
    96 return F.mse_loss(pred, noise)

Cell In[45], line 69, in DiffusionCond.forward(self, x_concat, t, cls, plane,
delta_t)
    66 c_proj = self.cond_proj(c_emb) # [B, time_embed_dim]
    68 # Call UNet - conditioning is implicit in concatenated input
--> 69 noise = self.unet(x_concat, t).sample
    70 return noise

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
module.py:1736, in Module._wrapped_call_impl(self, *args, **kwargs)
    1734     return self._compiled_call_impl(*args, **kwargs) # type:
ignore[misc]
    1735 else:
-> 1736     return self._call_impl(*args, **kwargs)

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
module.py:1747, in Module._call_impl(self, *args, **kwargs)
    1742 # If we don't have any hooks, we want to skip the rest of the logic in
    1743 # this function, and just call forward.
    1744 if not (self._backward_hooks or self._backward_pre_hooks or self.
_forward_hooks or self._forward_pre_hooks
    1745         or _global_backward_pre_hooks or _global_backward_hooks
    1746         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1747     return forward_call(*args, **kwargs)
    1749 result = None
    1750 called_always_called_hooks = set()

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
unets/unet_2d.py:320, in UNet2DModel.forward(self, sample, timestep,
class_labels, return_dict)
    316     sample, res_samples, skip_sample = downsample_block(
    317         hidden_states=sample, temb=emb, skip_sample=skip_sample

```

```

318         )
319     else:
--> 320         sample, res_samples = _
↳ downsample_block(hidden_states=sample, temb=emb)
322         down_block_res_samples += res_samples
324 # 4. mid

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1736, in Module._wrapped_call_impl(self, *args, **kwargs)
1734     return self._compiled_call_impl(*args, **kwargs) # type: _
↳ ignore[misc]
1735 else:
-> 1736     return self._call_impl(*args, **kwargs)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1747, in Module._call_impl(self, *args, **kwargs)
1742 # If we don't have any hooks, we want to skip the rest of the logic in
1743 # this function, and just call forward.
1744 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks
1745         or _global_backward_pre_hooks or _global_backward_hooks
1746         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1747     return forward_call(*args, **kwargs)
1749 result = None
1750 called_always_called_hooks = set()

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
↳ unets/unet_2d_blocks.py:1180, in AttnDownBlock2D.forward(self, hidden_states,
↳ temb, upsample_size, cross_attention_kwargs)
1178     else:
1179         hidden_states = resnet(hidden_states, temb)
-> 1180         hidden_states = attn(hidden_states, **cross_attention_kwargs)
1181         output_states = output_states + (hidden_states,)
1183 if self.downsamplers is not None:

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1736, in Module._wrapped_call_impl(self, *args, **kwargs)
1734     return self._compiled_call_impl(*args, **kwargs) # type: _
↳ ignore[misc]
1735 else:
-> 1736     return self._call_impl(*args, **kwargs)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1747, in Module._call_impl(self, *args, **kwargs)
1742 # If we don't have any hooks, we want to skip the rest of the logic in
1743 # this function, and just call forward.
1744 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks

```

```

1745         or _global_backward_pre_hooks or _global_backward_hooks
1746         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1747     return forward_call(*args, **kwargs)
1749 result = None
1750 called_always_called_hooks = set()

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
attention_processor.py:588, in Attention.forward(self, hidden_states,
encoder_hidden_states, attention_mask, **cross_attention_kwargs)
    583         logger.warning(
    584             f"cross_attention_kwargs {unused_kwargs} are not expected by
-> {self.processor.__class__.__name__} and will be ignored."
    585         )
    586 cross_attention_kwargs = {k: w for k, w in cross_attention_kwargs.
-> items() if k in attn_parameters}
-> 588 return self.processor(
    589     self,
    590     hidden_states,
    591     encoder_hidden_states=encoder_hidden_states,
    592     attention_mask=attention_mask,
    593     **cross_attention_kwargs,
    594 )

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
attention_processor.py:3286, in AttnProcessor2_0.__call__(self, attn,
encoder_hidden_states, attention_mask, temb, *args, **kwargs)
    3282         key = attn.norm_k(key)
    3284 # the output of sdp = (batch, num_heads, seq_len, head_dim)
    3285 # TODO: add support for attn.scale when we move to Torch 2.1
-> 3286 hidden_states = F.scaled_dot_product_attention(
    3287     query, key, value, attn_mask=attention_mask, dropout_p=0.0, is_causal=False
    3288 )
    3290 hidden_states = hidden_states.transpose(1, 2).reshape(batch_size, -1,
-> attn.heads * head_dim)
    3291 hidden_states = hidden_states.to(query.dtype)

```

```

OutOfMemoryError: CUDA out of memory. Tried to allocate 16.00 GiB. GPU 0 has a
total capacity of 15.92 GiB of which 0 bytes is free. Including non-PyTorch
memory, this process has 17179869184.00 GiB memory in use. Of the allocated
memory 20.18 GiB is allocated by PyTorch, and 26.75 MiB is reserved by PyTorch
but unallocated. If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation. See
documentation for Memory Management (https://pytorch.org/docs/stable/notes/
cuda.html#environment-variables)

```

```

[ ]: import matplotlib.pyplot as plt

@torch.no_grad()

```



```

def visualize_samples(num=4, plane='F0', use_classifier_guidance: bool = True):
    # Sample from val set for a specific plane
    plane_id = PLANE_TO_ID[plane]
    samp = val_df[val_df['plane'] == plane].sample(n=min(num,
    ↪len(val_df[val_df['plane'] == plane])))reset_index(drop=True)
    for i in range(len(samp)):
        row = samp.iloc[i]
        x_src = load_image(row['src_path']).to(device)[None]
        tgt_cls = torch.tensor([int(row['tgt_phase'])], device=device,
    ↪dtype=torch.long)
        plane_tensor = torch.tensor([plane_id], device=device, dtype=torch.long)
        delta_t = torch.tensor([int(row['delta_t']) - 1], device=device,
    ↪dtype=torch.long) # 0-indexed
        clf = phase_classifier if (use_classifier_guidance and phase_classifier
    ↪is not None and USE_CLASSIFIER_GUIDANCE) else None
        # If external classifier guidance is requested, temporarily enable grad
    ↪for sampling only
        if clf is not None:
            with torch.enable_grad():
                x_gen = diffusion.sample(shape=(1, 1, IMG_SIZE, IMG_SIZE),
    ↪cls=tgt_cls, plane=plane_tensor,
                delta_t=delta_t,
    ↪guidance_scale=GUIDANCE_SCALE, x_src=x_src, classifier=clf)
            else:
                # keep no-grad for pure sampling
                x_gen = diffusion.sample(shape=(1, 1, IMG_SIZE, IMG_SIZE),
    ↪cls=tgt_cls, plane=plane_tensor,
                delta_t=delta_t,
    ↪guidance_scale=GUIDANCE_SCALE, x_src=x_src, classifier=None)
        # show
        plt.figure(figsize=(10, 4))
        plt.subplot(1, 3, 1); plt.title(f'Source (x_t, {plane})'); plt.
    ↪imshow(denorm(x_src[0]).squeeze().cpu(), cmap='gray'); plt.axis('off')
        plt.subplot(1, 3, 2); plt.title(f'Predicted (x_{{t+Δ}})'); plt.
    ↪imshow(denorm(x_gen[0]).squeeze().cpu(), cmap='gray'); plt.axis('off')
        plt.subplot(1, 3, 3); plt.title(f'Difference'); plt.
    ↪imshow((denorm(x_gen[0]) - denorm(x_src[0])).squeeze().cpu().abs(),
    ↪cmap='hot'); plt.axis('off')
        plt.tight_layout()
        plt.show()

def save_tensor_image(tensor, path):
    arr = (denorm(tensor).clamp(0, 1).cpu().numpy() * 255).astype(np.uint8)
    if arr.ndim == 3 and arr.shape[0] == 1:
        arr = arr[0]

```

```

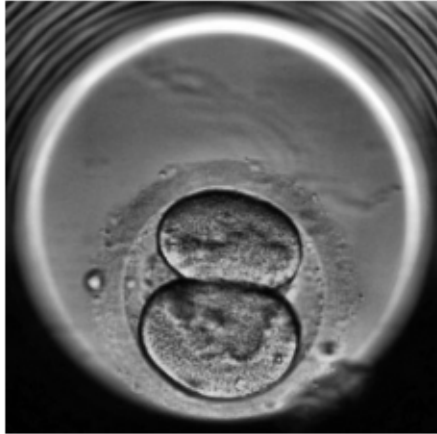
Image.fromarray(arr, mode='L').save(path)

@torch.no_grad()
def predict_next_frames(target_phase: str, plane: str, out_path: str,
    ↳ guidance_scale: float = GUIDANCE_SCALE, use_classifier_guidance: bool =
    ↳ True, delta_t: int = 2):
    assert target_phase in PHASE_LABELS, f"Unknown phase {target_phase}."
    ↳ Choices: {PHASE_LABELS}"
    assert plane in FOCAL_PLANES, f"Unknown plane {plane}. Choices:
    ↳ {FOCAL_PLANES}"
    cls = torch.tensor([PHASE_LABELS.index(target_phase)], device=device,
    ↳ dtype=torch.long)
    plane_id = torch.tensor([PLANE_TO_ID[plane]], device=device, dtype=torch.
    ↳ long)
    delta_t_tensor = torch.tensor([delta_t - 1], device=device, dtype=torch.
    ↳ long) # 0-indexed
    clf = phase_classifier if (use_classifier_guidance and phase_classifier is
    ↳ not None and USE_CLASSIFIER_GUIDANCE) else None
    if clf is not None:
        with torch.enable_grad():
            x_gen = diffusion.sample(shape=(1, 1, IMG_SIZE, IMG_SIZE), cls=cls,
    ↳ plane=plane_id,
                                delta_t=delta_t_tensor,
    ↳ guidance_scale=guidance_scale, x_src=None, classifier=clf)
        else:
            x_gen = diffusion.sample(shape=(1, 1, IMG_SIZE, IMG_SIZE), cls=cls,
    ↳ plane=plane_id,
                                delta_t=delta_t_tensor,
    ↳ guidance_scale=guidance_scale, x_src=None, classifier=None)
    out_path = str(out_path)
    save_tensor_image(x_gen[0], out_path)
    return out_path

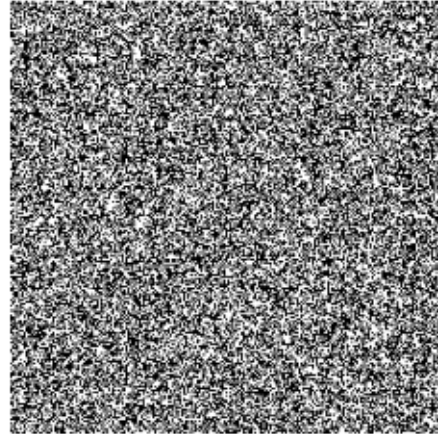
# Example after training:
# visualize_samples(4, plane='F0', use_classifier_guidance=True)

```

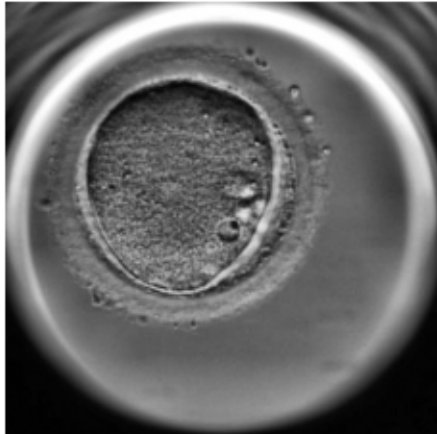
Source (x_t)



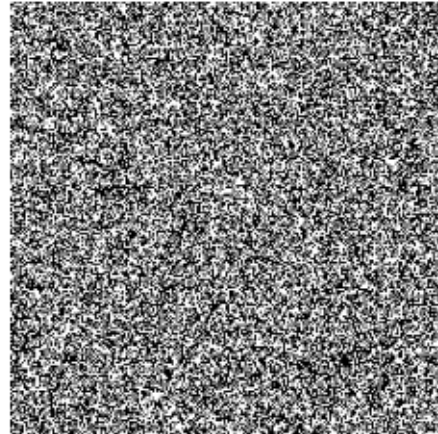
Predicted ($x_{t+\Delta}$)



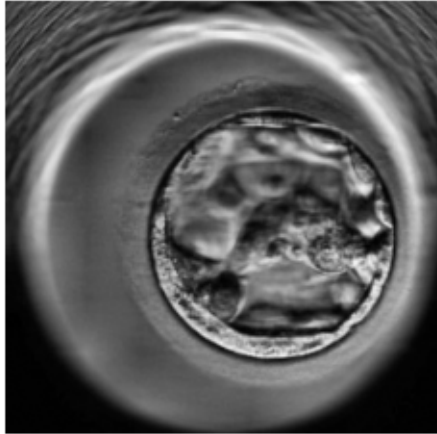
Source (x_t)



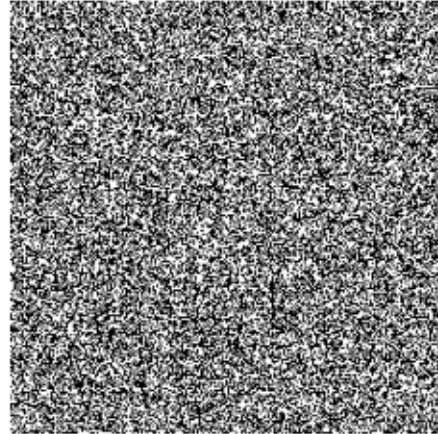
Predicted ($x_{t+\Delta}$)



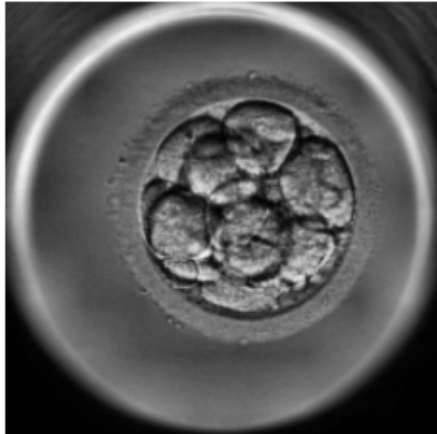
Source (x_t)



Predicted ($x_{t+\Delta}$)



Source (x_t)



Predicted ($x_{t+\Delta}$)

